

LAB REPORT

Title

CS3101: Operating System Lab 2 – Booting Unraveled

Name: Parth Singla **Roll Number:** 2401CS18

Objective :-

The primary objective of this assignment is to understand the exact sequence of low-level events that occur before a full operating system starts. Rather than building a complete operating system like Linux, this lab aims to build a small operating system step-by-step to practically demonstrate how the BIOS recognizes a boot sector, how raw disk sectors are loaded, how memory addresses are allocated, how the processor transitions from 16-bit real mode into 32-bit protected mode, and how a freestanding kernel communicates directly with hardware without the aid of standard software libraries.

Task 1: Boot Forensics

Task 1.1: Assembly Source Code

To begin the assignment, two small boot-sector programs were created in 16-bit mode with origin 0x7C00. Both initialize the segment registers and stack pointer, print a message, and are padded to exactly 512 bytes.

Source Code for **valid_boot.asm**:

```
1  bits 16      ; Start in 16-bit mode
2  org 0x7C00   ; Use origin 0x7C00
3
4  start:
5      cli      ; Disable interrupts before changing stack
6      xor ax, ax ; Clear AX
7      mov ds, ax ; Initialize DS to 0
8      mov es, ax ; Initialize ES to 0
9      mov ss, ax ; Initialize SS to 0
10     mov sp, 0x7C00 ; Initialize SP to 0x7C00
11     cld      ; Clear direction flag
12     sti      ; Re-enable interrupts
13
14     mov si, message ; Point SI to the start of the message
15     ;L for Agent
16 print_loop:
17     lodsb    ; Load the next byte from SI into AL
18     test al, al ; Check if AL is 0 (null terminator)
19     jz halt  ; If 0, jump to the halt routine
20     mov ah, 0x0E ; BIOS teletype output function
21     int 0x10 ; Call BIOS video service to print character
22     jmp print_loop ; Loop to the next character
23
24 halt:      ; Halt safely
25     cli      ; Disable interrupts
26     hlt      ; Halt the processor
27     jmp halt ; Permanent loop
28
29 message:
30     db "Welcome to Stage 1 Bootloader", 0 ; Short null-terminated message
31
32 ; Pad the file so that the final binary is exactly 512 bytes
33 times 510-($-$$) db 0
34
35 ; Valid final two-byte signature
36 dw 0xAA55
```

Source Code for corrupt_boot.asm: The corrupt bootloader contains the exact same instructions and padding, but the final two-byte signature differs.

```
1  bits 16
2  org 0x7C00
3
4  start:
5      cli
6      xor ax, ax
7      mov ds, ax
8      mov es, ax
9      mov ss, ax
10     mov sp, 0x7C00
11     cld
12     sti
13
14     mov si, message
15
16 print_loop:
17     lodsb
18     test al, al
19     jz halt
20     mov ah, 0x0E
21     int 0x10
22     jmp print_loop
23
24 halt:
25     cli
26     hlt
27     jmp halt
28
29 message:
30     db "Welcome to Stage 1 Bootloader", 0
31
32 ; Zero padding up to byte 510
33 times 510-($-$$) db 0
34
35 ; Incorrect final two-byte signature to make it corrupt
36 dw 0x0000
```

Task 1.2: Assembly and Verification

The source files were assembled into flat binaries using NASM. Using the `ls -l` command, both resulting files were verified to be exactly 512 bytes.

```
[parthsingla@172-9-7-130 2401CS18_OSlab2 % nasm -f bin valid_boot.asm -o valid_boot.bin
[parthsingla@172-9-7-130 2401CS18_OSlab2 % nasm -f bin corrupt_boot.asm -o corrupt_boot.bin
[parthsingla@172-9-7-130 2401CS18_OSlab2 % ls -l valid_boot.bin corrupt_boot.bin
-rw-r--r--@ 1 parthsingla staff 512 5 Aug 15:31 corrupt_boot.bin
-rw-r--r--@ 1 parthsingla staff 512 5 Aug 15:31 valid_boot.bin
parthsingla@172-9-7-130 2401CS18_OSlab2 %
```

Task 1.3: Inspect the Bytes

A hexadecimal dump of `valid_boot.bin` was generated using `xxd`, and the binary was disassembled using `ndisasm`.

```
parthsingla@172-9-7-130 2401CS18_OSlab2 % xxd -g 1 -l 512 valid_boot.bin
00000000: fa 31 c0 8e d8 8e c0 8e d0 bc 00 7c fc fb be 20 .1.....|...
00000010: 7c ac 84 c0 74 06 b4 0e cd 10 eb f5 fa f4 eb fc |...t.....
00000020: 57 65 6c 63 6f 6d 65 20 74 6f 20 53 74 61 67 65 Welcome to Stage
00000030: 20 31 20 42 6f 6f 74 6c 6f 61 64 65 72 00 00 00 1 Bootloader...
00000040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000090: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000110: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000120: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000130: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000140: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000150: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000160: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000170: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000190: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000001f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 55 aa .....U.
parthsingla@172-9-7-130 2401CS18_OSlab2 % ndisasm -b 16 -o 0x000 valid_boot.bin | head -n 25
00000000 FA cli
00000001 31C0 xor ax,ax
00000003 8ED8 mov ds,ax
00000005 8EC0 mov es,ax
00000007 8ED0 mov ss,ax
00000009 BC07C mov sp,0x7c00
0000000C FC cld
0000000D FB sti
0000000E BE207C mov si,0x7c20
00000011 AC lodsb
00000012 84C0 test al,al
00000014 7406 jz 0x1c
00000016 B40E mov ah,0xe
00000018 CD10 int byte 0x10
0000001A EBF5 jmp 0x11
0000001C FA cli
0000001D F4 hlt
0000001E EBFC jmp 0x1c
00000020 57 push di
00000021 656C gs insb
00000023 636F6D arpl [bx+0x6d],bp
00000026 6520746F and [gs:si+0x6f],dh
0000002A 205374 and [bp+di+0x74],dl
0000002D 61 popa
0000002E 67652031 and [gs:ecx],dh
parthsingla@172-9-7-130 2401CS18_OSlab2 %
```

Region Analysis: Based on the hexadecimal dump, the binary file consists of four main regions:

- **Machine-code instructions:**
 - **Starting offset:** 0x0000
 - **Ending offset:** 0x001F
 - **Purpose:** Contains the executable instructions generated from the assembly code to initialize registers and print the string.
- **Text message:**
 - **Starting offset:** 0x0020
 - **Ending offset:** 0x003D
 - **Purpose:** Stores the ASCII characters for "Welcome to Stage 1 Bootloader", terminating with a 00 byte.
- **Zero padding:**
 - **Starting offset:** 0x003E
 - **Ending offset:** 0x01FD
 - **Purpose:** Repeated 00 bytes added by the assembler to ensure the final binary file is exactly 512 bytes long.
- **Boot signature:**
 - **Starting offset:** 0x01FE
 - **Ending offset:** 0x01FF
 - **Purpose:** The final two bytes (55 AA) required by the BIOS to recognize the sector as bootable.

Disassembly Comparison Table: The following table maps the generated machine code back to the original assembly instructions using offsets beginning from 0x0000:

File offset	Machine-codebytes	Assembly instruction	Meaning
0	FA	cli	Disable maskable interrupts
1	31 C0	xor ax, ax	Set AX to zero
3	8E D8	mov ds, ax	Copy zero into DS
9	BC 00 7C	mov sp, 0x7C00	Initialize the stack pointer
1D	F4	hlt	Halt the processor

Task 1.4: Predict and Test BIOS Behaviour

- **Prediction:** Before running the emulator, I predicted that `valid_boot.bin` would successfully boot and print the message because it contains the required `0xAA55` signature. I predicted `corrupt_boot.bin` would be rejected.
- **Observation:** When executed as raw disk images in QEMU, the valid image successfully displayed the text. The corrupt image failed with a "not a bootable disk" error

```
[parthsingla@172-9-7-130 2401CS18_OsLab2 % qemu-system-i386 -drive format=raw,file=valid_boot.bin
q
^Cqemu-system-i386: terminating on signal 2 from pid 58456 (<unknown process>)
[parthsingla@172-9-7-130 2401CS18_OsLab2 % qemu-system-i386 -drive format=raw,file=corrupt_boot.bin
^Cqemu-system-i386: terminating on signal 2 from pid 58456 (<unknown process>)
```

```
SeaBIOS (version rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org)

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FD1FB0+06F31FB0 CA00

Booting from Hard Disk...
Welcome to Stage 1 Bootloader_
```

```
SeaBIOS (version rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org)

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FD1FB0+06F31FB0 CA00
Boot failed: not a bootable disk

Booting from Floppy...
Boot failed: could not read the boot disk

Booting from DVD/CD... Is disk
Boot failed: Could not read from CDROM (code 0003)
Booting from ROM...
iPXE (PCI 00:03.0) starting execution...ok
iPXE initialising devices...ok

iPXE 1.20.1+ (g4bd0) -- Open Source Network Boot Firmware -- http://ipxe.org
Features: DNS HTTP iSCSI TFTP AoE ELF MB00T PXE bzimage Menu PXEXT

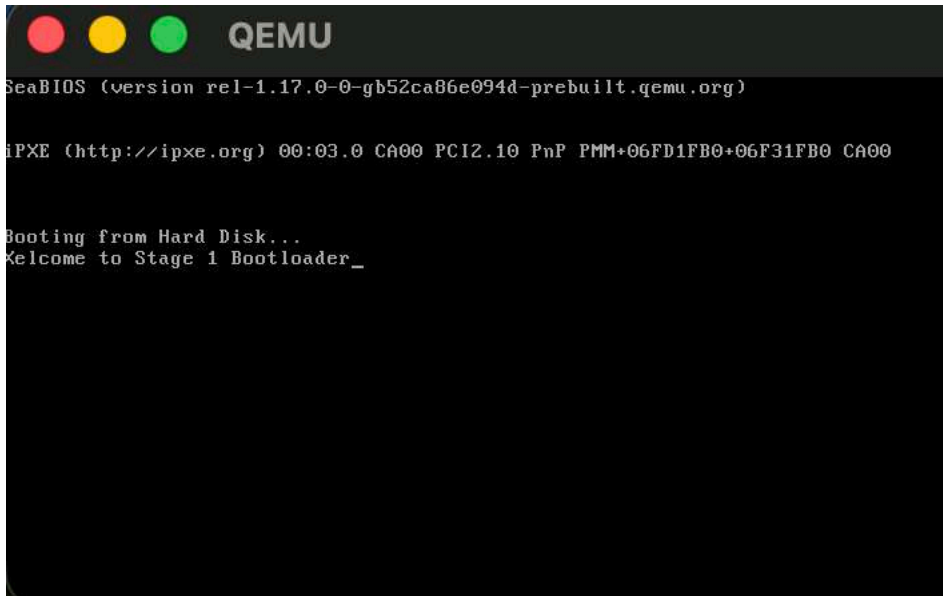
net0: 52:54:00:12:34:56 using 02540em on 0000:00:03.0 (open)
[Linkup, TX:0 TXE:0 RX:0 RXE:0]
Configuring (net0 52:54:00:12:34:56)..... ok
net0: 10.0.2.15/255.255.255.0 gw 10.0.2.2
Nothing to boot: No such file or directory (http://ipxe.org/2d03e13b)
No more network devices

No bootable device.
```

Explanation: The BIOS strictly dictates whether a 512-byte sector is bootable by checking the final two bytes. Because `corrupt_boot.bin` lacked the `55 AA` signature, the BIOS assumed it did not contain a valid bootloader and refused to transfer processor control to it, resulting in the boot failure.

Task 1.5: Modify the Compiled Binary Directly

Using the `dd` command, a single character in the compiled message payload was patched directly (replacing 'W' with 'X' at offset 32), modifying the file without reassembling the source code.



```
[parthsingla@172-9-7-130 2401CS18_OSlab2 % cp valid_boot.bin patched_boot.bin
[parthsingla@172-9-7-130 2401CS18_OSlab2 % printf 'X' | dd of=patched_boot.bin bs=1 seek=32 count=1 conv=notrunc
1+0 records in
1+0 records out
1 bytes transferred in 0.000075 secs (13333 bytes/sec)
[parthsingla@172-9-7-130 2401CS18_OSlab2 % strings patched_boot.bin
Welcome to Stage 1 Bootloader
```

Explanation: Altering a message byte simply changes the static data payload that the machine instructions read and print to the screen. It does not corrupt the instructions or the file size. Most importantly, it leaves the boot signature intact. Changing the final signature bytes, however, destroys bootability because the BIOS relies entirely on those specific bytes for hardware-level validation.

Task 2: Reusable Real-Mode Console

Source Code for `console.asm`:

A single assembly file, `console.asm`, was created to initialize the processor state, implement reusable output procedures, and display structured diagnostic information

```
1  bits 16
2  org 0x7C00
3
4  start:
5      ; Task 2.1 - Initialize the processor state
6      ; Immediately preserve the BIOS boot-drive number from DL
7      mov [boot_drive], dl
8
9      ; Disable maskable interrupts while changing the stack
10     cld
11     xor ax, ax
12     mov ds, ax      ; Set DS to zero
13     mov es, ax      ; Set ES to zero
14     mov ss, ax      ; Set SS to zero
15     mov sp, 0x7C00  ; Initialize SP to 0x7C00
16     cld             ; Clear the direction flag
17     sti             ; Re-enable interrupts
18
19     ; Task 2.3 - Produce structured diagnostic output
20     ; Print Line 1
21     mov si, msg_welcome
22     call print_string
23     call print_newline
24
25     ; Print Line 2
26     mov si, msg_studentid
27     call print_string
28     call print_newline
29
30     ; Print Line 3
31     mov si, msg_bootdrive
32     call print_string
33     mov al, [boot_drive]
34     call print_hex8
35     call print_newline
36
37     halt_loop:
38     ; Permanent safe loop containing cli, hlt, and a jump
39     cli
40     hlt
41     jmp halt_loop
42
43     ; print_char: Prints character in AL
44     print_char:
45     push ax
46     push bx
47     mov ah, 0x0E      ; BIOS teletype output
48     xor bh, bh        ; Page number 0
49     int 0x10          ; Only place int 0x10 appears
50     pop bx
51     pop ax
52     ret
53
54     ; print_string: Prints null-terminated string at DS:SI
55     print_string:
56     push ax
57     push si
58     .loop:
59     lodsb             ; Load next byte into AL
60     test al, al        ; Check for null terminator (00 byte)
61     jz .done
62     call print_char ; Call print_char instead of int 0x10
63     jmp .loop
64     .done:
65     pop si
66     pop ax
67     ret
68
69     ; print_newline: Prints carriage-return (0x0D) and line-feed (0x0A)
70     print_newline:
71     push ax
72     mov al, 0x0D
73     call print_char
74     mov al, 0x0A
75     call print_char
76     pop ax
77     ret
78
79     ; print_hex8: Prints 8-bit value in AL as two uppercase hex digits
80     print_hex8:
81     push ax
82     push cx
83     mov ah, al      ; Save original value to AH
84
85     ; Extract and print high 4-bit nibble
86     shr al, 4
87     call print_hex_digit
88
89     ; Extract and print low 4-bit nibble
90     mov al, ah
91     and al, 0x0F
92     call print_hex_digit
93
94     pop cx
95     pop ax
96     ret
97
98     ; Helper procedure for print_hex8
99     print_hex_digit:
100    push ax
101    cmp al, 9
102    jle .is_digit
103    add al, 'A' - 10 ; Convert 10-15 to A-F
104    jmp .print
105    .is_digit:
106    add al, '0'      ; Convert 0-9 to ASCII '0'-'9'
107    .print:
108    call print_char
109    pop ax
110    ret
111
112    boot_drive:  db 0
113    msg_welcome:  db "Welcome to Real-Mode Console", 0
114    msg_studentid: db "Student ID: 2401CS18", 0
115    msg_bootdrive: db "Boot drive: 0x", 0
116
117    ; Boot sector padding and signature
118    times 510-($-$$) db 0
119    dw 0xAA55
```

Task 2.1: Safety of Processor State Initialization

Modifying the stack segment (SS) and stack pointer (SP) registers while maskable interrupts are enabled is unsafe[cite: 1]. Setting up a stack is not a single atomic operation; it requires updating SS and SP sequentially. If a hardware interrupt occurs precisely after SS is modified but before SP is updated, the CPU will push the return address and flags to a mismatched, fragmented memory address. This memory corruption typically results in a system crash. Disabling maskable interrupts

(`cli`) before the modifications and re-enabling them (`sti`) afterward guarantees the stack is transitioned safely

Task 2.2 and 2.3: Compilation, Execution, and Output

The source file was compiled using NASM into a flat binary and booted via QEMU. The procedures correctly outputted the requested strings and performed the necessary hexadecimal conversions

```
[parthsingla@172-9-7-130 2401CS18_OSlab2 % nasm -f bin console.asm -o console.bin  
[parthsingla@172-9-7-130 2401CS18_OSlab2 % qemu-system-i386 -drive format=raw,file=console.bin
```

displays the terminal commands used to compile `console.asm` into `console.bin` and launch the QEMU emulator.

```
SeaBIOS (version rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org)  
  
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FD1FB0+06F31FB0 CA00  
  
Booting from Hard Disk...  
Welcome to Real-Mode Console  
Student ID: 2401CS18  
Boot drive: 0x80
```

The system successfully prints the three required lines of diagnostic output, including the custom Student ID and the preserved boot drive properly formatted as the hexadecimal value `0x80`

Task 3.1 and 3.2: Assembly Source Code

To transition from a 16-bit real mode environment to 32-bit protected mode, a robust multi-stage bootloader was developed

Source Code for `stage1.asm`: Stage 1 is a 512-byte boot sector that uses a Disk Address Packet (DAP) and BIOS INT 13h extended read service to load Stage 2 into memory at physical address 0x8000. It implements a three-attempt retry policy and validates that the loaded data begins with the ASCII magic string `STG2`

```
1  bits 16
2  org 0x7C00
3
4  start:
5      ; Preserve BIOS boot-drive number from DL immediately
6      mov [boot_drive], dl
7
8      cli
9      xor ax, ax
10     mov ds, ax
11     mov es, ax
12     mov ss, ax
13     mov sp, 0x7C00
14     cld
15     sti
16
17     mov si, msg_stage1
18     call print_string
19
20     ; Load Stage 2: Max 3 attempts
21     mov cx, 3
22 load_stage2_loop:
23     push cx
24
25     ; Use BIOS INT 13h extended read service AH=0x42
26     mov ah, 0x42
27     mov dl, [boot_drive]
28     mov si, dap_stage2
29     int 0x13
30
31     jnc validate_stage2 ; If carry clear, read successful
32
33     ; If read fails, reset disk controller and try again
34     xor ah, ah
35     mov dl, [boot_drive]
36     int 0x13
37
38     pop cx
39     loop load_stage2_loop
40
41     ; If all 3 attempts fail, print error and halt
42     mov si, msg_err_read
43     call print_string
44     jmp halt
45
46 validate_stage2:
47     pop cx
48     ; Compare first four bytes at 0x8000 with ASCII STG2
49     ; Note: x86 is little-endian, so 'STG2' is 0x32475453
50     mov eax, dword [0x8000]
51     cmp eax, 0x32475453
52     je stage2_validated
53
54     mov si, msg_err_val
55     call print_string
```

```
56     jmp halt
57
58 stage2_validated:
59     ; Transfer control to Stage 2 at the documented entry location
60     jmp 0x0000:0x8004
61
62 halt:
63     cli
64     hlt
65     jmp halt
66
67 ; Minimal print string function for bootloader
68 print_string:
69     push ax
70     push si
71 .loop:
72     lodsb
73     test al, al
74     jz .done
75     mov ah, 0x0E
76     int 0x10
77     jmp .loop
78 .done:
79     pop si
80     pop ax
81     ret
82
83 ; Data Section
84 boot_drive:  db 0
85 msg_stage1:  db "Stage 1 started...", 0x0D, 0x0A, 0
86 msg_err_read: db "Disk read failed!", 0
87 msg_err_val:  db "Stage 2 validation failed!", 0
88
89 ; 16-byte Disk Address Packet (DAP)
90 dap_stage2:
91     db 0x10          ; Size of DAP
92     db 0             ; Reserved
93     dw 4              ; Read exactly four sectors
94     dw 0x8000        ; Offset
95     dw 0x0000        ; Segment
96     dq 1              ; Beginning at LBA 1
97
98 times 510-($-$$) db 0
99 dw 0xAA55
```

Source Code for `stage2.asm`: Stage 2 begins with the `STG2` header, reinitializes the segment registers, and reads 32 sectors starting from LBA 5 into physical address `0x10000`. It validates the `KRNL` signature, enables the `A20` line, loads a Global Descriptor Table (GDT), sets the `PE` bit in `CR0`, and performs a far jump to enter 32-bit protected mode

```

1  bits 16
2  org 0x0000
3
4  ; The first four bytes of stage2.bin must be ASCII STG2
5  db 'S', 'T', 'G', '2'
6
7  stage2_entry:
8      ; Reinitialize DS, ES, SS, and SP
9      cli
10     xor ax, ax
11     mov ds, ax
12     mov es, ax
13     mov ss, ax
14     mov sp, 0x0000
15     cld
16     sti
17
18     mov si, msg_stage2
19     call print_string
20
21     ; Read 32 sectors beginning at LBA 5 to physical address 0x10000
22     ; (Physical 0x10000 = Segment 0x1000, Offset 0x0000)
23     mov cx, 32
24     load_kernel_loop:
25         push cx
26         mov ah, 0x42
27         mov dl, 0x00 ; Hardcoded for primary hard disk testing
28         mov si, dap_kernel
29         int 0x13
30         jnc validate_kernel
31
32         xor ah, ah ; Reset on failure
33         mov dl, 0x00
34         int 0x13
35
36         pop cx
37     loop load_kernel_loop
38     jmp halt
39
40 validate_kernel:
41     pop cx
42     ; Compare first four bytes at 0x10000 with ASCII KRNL
43     ; Use segment 0x1000 to access physical 0x10000 in real mode
44     mov ax, 0x1000
45     mov es, ax
46     mov eax, dword [es:0x0000]
47     cmp eax, 0x4C45524B ; Little-endian for 'KRNL'
48     je kernel_validated
49     jmp halt
50
51 kernel_validated:
52     mov si, msg_kernel_ok
53     call print_string
54
55     ; Enter 32-bit protected mode
56     ; Enable A20 line (Fast A20)
57     in al, 0x92
58     or al, 2
59     out 0x92, al
60
61     ; Load the GDT with LGDT
62     cli
63     lgdt [gdt_descriptor]
64
65     ; Set the PE bit in CR0
66     mov eax, cr0
67     or eax, 0x1
68     mov cr0, eax
69
70     ; Use a far jump to load protected-mode code selector and clear prefetch queue
71     jmp 0x00:init_pm
72
73 ; 32-bit Mode Execution
74 bits 32
75 init_pm:
76     ; Load data selector into DS, ES, FS, GS, and SS
77     mov ax, 0x10
78     mov ds, ax
79     mov es, ax
80     mov fs, ax
81     mov gs, ax
82
83     mov ss, ax
84
85     ; Set ESP to 0x00000
86     mov esp, 0x00000
87
88     ; Jump to the kernel entry at physical address 0x10004
89     jmp 0x10004
90
91 ; 16-bit Print Helper
92 bits 16
93 print_string:
94     push ax
95     push si
96 .loop:
97     lodsb
98     test al, al
99     jz .done
100    mov ah, 0x0E
101    int 0x10
102    jmp .loop
103 .done:
104    pop si
105    pop ax
106    ret
107
108 halt:
109     cli
110     hlt
111     jmp halt
112
113 ; Data
114 msg_stage2: db "Stage 2 loaded and validated", 0x0D, 0x0A, 0
115 msg_kernel_ok: db "Kernel loaded and validated", 0x0D, 0x0A, 0
116
117 dap_kernel:
118     db 0x10
119     db 0
120     dw 32 ; 32 sectors
121     dw 0x0000 ; Offset 0
122     dw 0x1000 ; Segment 0x1000 (resolves to 0x10000 physical)
123     dq 5 ; LBA 5
124
125 ; GDT Definition containing null, code, and data descriptors
126 gdt_start:
127     ; Null descriptor
128     dd 0x0
129     dd 0x0
130     ; Flat 32-bit code descriptor
131     dw 0xFFFF
132     dw 0x0
133     db 0x0
134     db 10011010b
135     db 11001111b
136     db 0x0
137     ; Flat 32-bit data descriptor
138     dw 0xFFFF
139     dw 0x0
140     db 0x0
141     db 10010010b
142     db 11001111b
143     db 0x0
144 gdt_end:
145
146 gdt_descriptor:
147     dw gdt_end - gdt_start - 1
148     dd gdt_start
149
150 ; Pad Stage 2 to exactly 4 sectors (2048 bytes)
151 times 2048-($-$$) db 0

```

Task 3 Execution and Verification

To test the multi-stage loader, a temporary 32-sector `dummy_kernel.asm` was created with the required KRNL header. The three binaries were assembled and concatenated together into a single raw disk image (`os_image.bin`) ensuring they aligned perfectly with their designated Logical Block Addresses (LBA).

```
parthsingla@172-9-7-130 2401CS18_OSlab2 % nasm -f bin stage1.asm -o stage1.bin
nasm -f bin stage2.asm -o stage2.bin
[nasm -f bin dummy_kernel.asm -o kernel.bin
[parthsingla@172-9-7-130 2401CS18_OSlab2 % cat stage1.bin stage2.bin kernel.bin > os_image.bin
[parthsingla@172-9-7-130 2401CS18_OSlab2 % qemu-system-i386 -drive format=raw,file=os_image.bin
```

displays the terminal commands used to compile the three assembly files, concatenate them using the `cat` command, and launch the QEMU emulator.

```
SeaBIOS (version rel-1.17.0-0-gb52ca86e094d-prebuilt.qemu.org)

iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FD1FB0+06F31FB0 CA00

Booting from Hard Disk...
Stage 1 started...
Stage 2 loaded and validated
Kernel loaded and validated
```

demonstrates the successful execution in QEMU. The output strictly proves that Stage 1 successfully loaded and validated Stage 2, and subsequently, Stage 2 successfully loaded and validated the kernel from the disk

Task 4: Build a 32-bit Protected-Mode Mini Kernel

Task 4.2 and 4.7: Kernel Entry and Linker Script

To properly format the kernel in memory, a custom linker script (`linker.ld`) was created to ensure the kernel image begins at physical address `0x10000` with the KRNL header occupying the first four bytes. The `kernel_entry` was placed at `0x10004`.

Source Code for `linker.ld`:

```
1 ENTRY(kernel_entry)
2 SECTIONS
3 {
4     /* 1. Kernel image begins at physical address 0x10000 */
5     . = 0x10000;
6
7     /* 2. The KRNL header occupies exactly four bytes at 0x10000 */
8     .header : {
9         *(.header)
10    }
11
12    /* 3. kernel_entry begins at address 0x10004 */
13    . = 0x10004;
14
15    /* 4. Entry assembly code and remaining text */
16    .text : {
17        *(.text)
18    }
19
20    /* 5. Place read-only data, data, and uninitialized data (BSS) */
21    .rodata : { *(.rodata) }
22    .data : { *(.data) }
23
24    .bss : {
25        __bss_start = .;
26        *(COMMON)
27        *(.bss)
28        __bss_end = .;
29    }
30
31    /* 6. Discard unnecessary hosted-runtime sections */
32    /DISCARD/ : {
33        *(.eh_frame)
34        *(.comment)
35        *(.note.GNU-stack)
36    }
37 }
```

Source Code for `kernel_entry.asm`:

This assembly file defines the KRNL header, clears the direction flag, manually zeroes out the `.bss` section (as no standard C runtime is available to do this), and calls the C function `kernel_main`

```
1 bits 32
2 global kernel_entry
3 extern kernel_main
4 extern __bss_start
5 extern __bss_end
6
7 section .header
8     ; KRNL header at 0x10000
9     db 'K', 'R', 'N', 'L'
10
11 section .text
12 kernel_entry:
13     cld ; Clear the direction flag
14
15     ; Clear the complete .bss section
16     mov edi, __bss_start
17     mov ecx, __bss_end
18     sub ecx, edi
19     shr ecx, 2 ; Divide by 4 for 32-bit {dword} blocks
20     xor eax, eax
21     rep stosd ; Fill .bss with zeros
22
23     ; Call the C function kernel_main
24     call kernel_main
25
26 halt_loop:
27     ; Halt safely if kernel_main unexpectedly returns
28     cli
29     hlt
30     jmp halt_loop
```

Task 4.3, 4.4, 4.5, and 4.6: Freestanding C Kernel and Shell

A freestanding C kernel was implemented without hosted-library facilities. It writes directly to the VGA text memory at 0xB8000 to support screen clearing, scrolling, color manipulation, and a hardware cursor. Keyboard input is handled by polling the PS/2 controller at status port 0x64 and data port 0x60, translating Scan-Code Set 1 values, and storing them in a bounded command buffer. Finally, it implements a command parser for a custom shell environment.

Source Code for `kernel.c`:

```

1 typedef unsigned char uint8_t;
2 typedef unsigned short uint16_t;
3 typedef unsigned int uint32_t;
4
5 // VGA Constants
6 #define VGA_ADDR 0xB8000
7 #define VGA_COLS 80
8 #define VGA_ROWS 25
9
10 // State variables
11 uint16_t* vga_buffer = (uint16_t*)VGA_ADDR;
12 int term_col = 0;
13 int term_row = 0;
14 uint8_t user_color = 0x0F; // Track the user's chosen text color
15 uint8_t term_color = 0x0F; // White text (F) on black background (0)
16
17 void term_print(const char* str);
18 void term_clear(void);
19 void term_putchar(char c);
20 void term_set_color(uint8_t fg);
21
22 // I/O Port Helper for the Hardware Cursor
23 static inline void outb(uint16_t port, uint8_t val) {
24     __asm__ volatile ("outb %B, %I" : : "a"(val), "Nd"(port));
25 }
26
27 // Port input helper for reading keyboard data
28 static inline uint8_t inb(uint16_t port) {
29     uint8_t ret;
30     __asm__ volatile ("inb %I, %A" : "=a"(ret) : "Nd"(port));
31     return ret;
32 }
33
34 int strcmp(const char* s1, const char* s2) {
35     while (*s1 && *s1 == *s2) { s1++; s2++; }
36     return *(const unsigned char*)s1 - *(const unsigned char*)s2;
37 }
38
39 int atoi(const char* str) {
40     int res = 0;
41     while (*str >= '0' && *str <= '9') {
42         res = res * 10 + (*str - '0');
43         str++;
44     }
45     return res;
46 }
47
48 uint32_t parse_hex(const char* str) {
49     uint32_t val = 0;
50     if (str[0] == '0' && (str[1] == 'x' || str[1] == 'X')) str += 2;
51     while (*str) {
52         val <<= 4;
53         if (*str >= '0' && *str <= '9') val |= (*str - '0');
54         else if (*str >= 'a' && *str <= 'f') val |= (*str - 'a' + 10);
55         else if (*str >= 'A' && *str <= 'F') val |= (*str - 'A' + 10);
56         str++;
57     }
58     return val;
59 }
60
61 void print_hex32(uint32_t val) {
62     const char hex_digits[] = "0123456789ABCDEF";
63     for (int i = 28; i >= 0; i -= 4) {
64         term_putchar(hex_digits[(val >> i) & 0xF]);
65     }
66 }
67
68 #define CMD_MAX 128
69 char cmd_buffer[CMD_MAX]; // Bounded buffer to prevent overflow
70 int cmd_len = 0;
71 int shift_pressed = 0;
72
73 const char scancode_chars[] = {
74     0, 0, '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', '-', '=', '\b',
75     '\t', '[', '\\', ']', '^', '_', '`', '{', '|', '~', ' ', '\n',
76     0, 'a', 's', 'd', 'f', 'g', 'h', 'j', 'k', 'l', ';', '\'', '/', '>', ' ',
77     0, '\\', 'z', 'x', 'c', 'v', 'b', 'n', 'm', ',', '.', '<', '>', ' ', ' ',
78 };

```

```

70 const char scancode_chars_shift[] = {
71     0, 0, '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', 'q', 'w', 'e', 'r', 't', 'y', 'u', 'i', 'o', 'p', 'a', 's', 'd', 'f', 'g', 'h', 'j', 'k', 'l', 'z', 'x', 'c', 'v', 'b', 'n', 'm', '.,', '/', '<';
72     '\t', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', 'p', 'a', 's', 'd', 'f', 'g', 'h', 'j', 'k', 'l', 'z', 'x', 'c', 'v', 'b', 'n', 'm', '.,', '/', '<';
73     0, 'A', 'S', 'D', 'F', 'G', 'H', 'J', 'K', 'L', 'Z', 'X', 'C', 'V', 'B', 'N', 'M', '.,', '/', '<';
74     0, '1', '2', '3', '4', '5', '6', '7', '8', '9', '0', 'p', 'a', 's', 'd', 'f', 'g', 'h', 'j', 'k', 'l', 'z', 'x', 'c', 'v', 'b', 'n', 'm', '.,', '/', '<';
75 };
76
77 // Command Parser
78 void execute_command() {
79     if (cmd_len == 0) return; // Empty commands must not crash the kernel
80
81     char cmd[CMD_MAX], arg[CMD_MAX];
82     int i = 0, j = 0;
83
84     // Separate the first word from the remaining argument text
85     while (i < cmd_len && cmd_buffer[i] != ' ') { cmd[i] = cmd_buffer[i]; i++; }
86     cmd[i] = '\0';
87
88     // Skip spaces between command and argument
89     while (i < cmd_len && cmd_buffer[i] == ' ') i++;
90
91     // Copy the rest into arg
92     while (i < cmd_len) { arg[j++] = cmd_buffer[i++]; }
93     arg[j] = '\0';
94
95     if (strcmp(cmd, "help") == 0) {
96         // Display all supported commands and their syntax
97         term_print("help          show this command list\n");
98         term_print("clear          clear the screen\n");
99         term_print("echo <text>    print text\n");
100        term_print("info           show OS information\n");
101        term_print("regs          show CR0 and ESP\n");
102        term_print("color <0-15>  set foreground color\n");
103        term_print("peek <address> read one memory byte\n");
104        term_print("reboot        reboot through the 8842\n");
105        term_print("halt          stop the CPU\n");
106    } else if (strcmp(cmd, "clear") == 0) {
107        term_clear(); // Clear the VGA screen and return cursor to top-left
108    } else if (strcmp(cmd, "echo") == 0) {
109        term_print(arg); // Display the complete argument string
110        term_putchar('\n');
111    } else if (strcmp(cmd, "info") == 0) {
112        // Display OS name, version, processor mode, and console method
113        term_print("StudentsOS 1.0 (32-bit protected-mode, VGA console)\n");
114    } else if (strcmp(cmd, "regs") == 0) {
115        // Display the current values of CR0 and ESP
116        uint32_t cr0_val, esp_val;
117        __asm__ volatile("mov %cr0, %0" : "=r"(&cr0_val));
118        __asm__ volatile("mov %esp, %0" : "=r"(&esp_val));
119        term_print("CR0: 0x"); print_hex32(cr0_val); term_print("\n");
120        term_print("ESP: 0x"); print_hex32(esp_val); term_print("\n");
121    } else if (strcmp(cmd, "color") == 0) {
122        if (arg[0] == '\0') {
123            term_print("Usage: color <0-15>\n"); // Usage message for invalid values
124        } else {
125            int col = atoi(arg);
126            if (col >= 0 && col <= 15) {
127                user_color = (uint8_t)col; // Save the user's choice
128                term_set_color(user_color); // Apply it
129            }
130            else term_print("Invalid color. Use 0-15.\n"); // Usage message for invalid values
131        }
132    } else if (strcmp(cmd, "peek") == 0) {
133        if (arg[0] == '\0') {
134            term_print("Usage: peek <address>\n");
135        } else {
136            uint32_t addr = parse_hex(arg);
137            uint8_t val = *(uint8_t*)addr; // Read one byte from that memory address
138
139            // Display the byte in hexadecimal
140            term_print("Byte: 0x");
141            const char hex[] = "0123456789ABCDEF";
142            term_putchar(hex[val >> 4]);
143            term_putchar(hex[val & 0xF]);
144            term_putchar('\n');
145        }
146    }
147 }

```

Continued on next page...

```

155 } else if (strcmp(cmd, "reboot") == 0) {
156     // Request a processor reset through the 8042 keyboard controller
157     uint8_t temp;
158     do {
159         temp = inb(0x64);
160         if ((temp & 0x01) != 0) inb(0x60); // Clear keyboard buffer
161     } while ((temp & 0x02) != 0);
162     outb(0x64, 0xFE); // Pulse reset line
163     __asm__ volatile("cli; hlt"); // Fallback halt
164 } else if (strcmp(cmd, "halt") == 0) {
165     // Disable interrupts and halt the processor permanently
166     term_print("System halted.\n");
167     __asm__ volatile("cli; hlt");
168 } else {
169     // Informative message for an unsupported command
170     term_print("Unknown command: ");
171     term_print(cmd);
172     term_putchar('\n');
173 }
174 }
175
176 // B. Update Hardware Cursor
177 void update_cursor(int col, int row) {
178     uint16_t pos = row * VGA_COLS + col;
179     outb(0x3D4, 0x8F);
180     outb(0x3D5, (uint8_t)(pos & 0xFF));
181     outb(0x3D4, 0x8E);
182     outb(0x3D5, (uint8_t)((pos >> 8) & 0xFF));
183 }
184
185 // 1. Clear Screen
186 void term_clear() {
187     for (int y = 0; y < VGA_ROWS; y++) {
188         for (int x = 0; x < VGA_COLS; x++) {
189             vga_buffer[y * VGA_COLS + x] = (term_color << 8) | ' ';
190         }
191     }
192     term_col = 0;
193     term_row = 0;
194     update_cursor(term_col, term_row);
195 }
196
197 // 6. Upward Scrolling
198 void term_scroll() {
199     if (term_row >= VGA_ROWS) {
200         // Move everything up one row
201         for (int y = 1; y < VGA_ROWS; y++) {
202             for (int x = 0; x < VGA_COLS; x++) {
203                 vga_buffer[(y - 1) * VGA_COLS + x] = vga_buffer[y * VGA_COLS + x];
204             }
205         }
206         // Clear the bottom row
207         for (int x = 0; x < VGA_COLS; x++) {
208             vga_buffer[(VGA_ROWS - 1) * VGA_COLS + x] = (term_color << 8) | ' ';
209         }
210         term_row = VGA_ROWS - 1;
211     }
212 }
213
214 // 2, 4, 5. Print a single character with wrap and newline logic
215 void term_putchar(char c) {
216     if (c == '\n' || c == '\r') {
217         term_col = 0;
218         term_row++;
219     } else if (c == '\b') { // 7. Backspace erasure
220         if (term_col > 0) {
221             term_col--;
222             vga_buffer[term_row * VGA_COLS + term_col] = (term_color << 8) | ' ';
223         }
224     } else {
225         vga_buffer[term_row * VGA_COLS + term_col] = (term_color << 8) | c;
226         term_col++;
227         if (term_col >= VGA_COLS) { // Wrap line
228             term_col = 0;
229             term_row++;
230         }
231     }
232 }

```

```

233     term_scroll();
234     update_cursor(term_col, term_row);
235 }
236
237 // 3. Print a null-terminated string
238 void term_print(const char* str) {
239     for (int i = 0; str[i] != '\0'; i++) {
240         term_putchar(str[i]);
241     }
242 }
243
244 // 8. Change Color
245 void term_set_color(uint8_t fg) {
246     term_color = fg;
247 }
248
249 // C Entry Point
250 void kernel_main(void) {
251     term_clear();
252
253     term_print("StudentOS 1.0\n");
254     term_print("32-bit protected-mode mini kernel\n");
255     term_print("Type 'help' to list commands.\n\n");
256
257     term_set_color(0x0A); // Light Green
258     term_print("student-os> ");
259     term_set_color(0x0F); // Back to White
260
261     while (1) {
262         // Check status port 0x64 to see if data is available
263         if (inb(0x64) & 1) {
264             uint8_t scancode = inb(0x60); // Read scan code from port 0x60
265
266             // Handle Shift events
267             if (scancode == 0x2A || scancode == 0x36) {
268                 shift_pressed = 1;
269             } else if (scancode == 0xAA || scancode == 0xB6) {
270                 shift_pressed = 0;
271             } else if (scancode & 0x80) {
272                 // Safely ignore unsupported key-release and extended codes
273             } else {
274                 char c = 0;
275                 if (scancode < sizeof(scancode_chars)) {
276                     c = shift_pressed ? scancode_chars_shift[scancode] : scancode_chars[scancode];
277                 }
278
279                 if (c == '\n') { // Enter key logic
280                     term_putchar('\n');
281                     cmd_buffer[cmd_len] = '\0'; // Final null terminator
282                     execute_command();
283                     cmd_len = 0; // Reset buffer
284
285                     term_set_color(0x0A); // Light Green for prompt
286                     term_print("student-os> ");
287                     term_set_color(user_color); // Restore the user's chosen color!
288                 } else if (c == '\b') { // Backspace logic
289                     if (cmd_len > 0) {
290                         cmd_len--;
291                         term_putchar('\b');
292                     }
293                 } else if (c != 0) { // Add typed character to buffer
294                     if (cmd_len < CMD_MAX - 1) {
295                         cmd_buffer[cmd_len++] = c;
296                         term_putchar(c);
297                     }
298                 }
299             }
300         }
301     }
302 }

```

Task 4 Execution and Command Verification

The kernel was compiled into a 32-bit freestanding ELF file using `i686-elf-gcc`, linked with `linker.ld`, converted to a raw binary padded to 16384 bytes, and concatenated with the bootloaders.

```

parthsingla@172-9-7-130 2401CS18_OS1ab2 % nasm -f elf32 kernel_entry.asm -o kernel_entry.o
parthsingla@172-9-7-130 2401CS18_OS1ab2 % i686-elf-gcc -m32 -ffreestanding -fno-pie -fno-stack-protector -fno-builtin -nostdlib -c kernel.c -o kernel.o
parthsingla@172-9-7-130 2401CS18_OS1ab2 % i686-elf-ld -T linker.ld kernel_entry.o kernel.o -o kernel.elf
i686-elf-ld: warning: kernel.elf has a LOAD segment with RWX permissions
parthsingla@172-9-7-130 2401CS18_OS1ab2 % i686-elf-objcopy -O binary kernel.elf kernel.bin
dd if=/dev/zero of=kernel.bin bs=1 seek=16384 count=0
0+0 records in
0+0 records out
0 bytes transferred in 0.000014 secs (0 bytes/sec)
parthsingla@172-9-7-130 2401CS18_OS1ab2 % cat stage1.bin stage2.bin kernel.bin > os_image.bin
parthsingla@172-9-7-130 2401CS18_OS1ab2 % qemu-system-i386 -drive file=os_image.bin,format=raw,index=0,media=disk

```

shows the cross-compilation pipeline, binary formatting, and launching of QEMU.

```
StudentOS 1.0
32-bit protected-mode mini kernel
Type 'help' to list commands.

student-os>
```

System Boot: This image demonstrates the kernel successfully clearing the screen and loading the student-os> interactive prompt.

The implemented command shell was systematically tested to verify the behavior of all required commands:

help Command: This image demonstrates the parser identifying the command and displaying the supported syntax list.

```
StudentOS 1.0
32-bit protected-mode mini kernel
Type 'help' to list commands.

student-os>

help
help      show this command list
clear     clear the screen
echo <text> print text
info      show OS information
regs      show CR0 and ESP
color <0-15> set foreground color
peek <address> read one memory byte
reboot    reboot through the 8042
halt      stop the CPU
student-os> _
```

echo Command: This image demonstrates the ability to separate the argument string from the command and reprint it correctly.

```
student-os> echo Hello OS
Hello OS
```

info Command: This image shows the accurate display of OS and processor mode information

```
student-os> info
StudentOS 1.0 (32-bit protected-mode, VGA console)
```

regs Command: This image confirms the kernel can extract and print the live values of the CR0 and ESP registers in hexadecimal format using inline assembly.

```
student-os> regs
CR0: 0x00000011
ESP: 0x0008FE90
```

peek Command: This image verifies direct memory access by reading address 0xB8000 (the first byte of the VGA buffer) and displaying its hexadecimal value

```
student-os> peek 0xB8000
Byte: 0x73
```

color Command: This image highlights the VGA driver dynamically altering the foreground color attributes while managing the interactive shell interface

```
StudentOS 1.0  
32-bit protected-mode mini kernel  
Type 'help' to list commands.
```

```
student-os> color 1  
student-os> color 10  
student-os> color 9  
student-os> color 15  
student-os> color 2  
student-os> color _
```

Results and Discussion

The assignment was divided into four sequential tasks, all of which were executed and validated successfully:

- **Boot Forensics (Task 1):** Two 512-byte boot sectors were developed. Emulation results confirmed that the BIOS successfully executes the valid bootloader containing the 0xAA55 signature but firmly rejects the corrupt bootloader. Hexadecimal inspection and direct binary patching demonstrated how machine instructions, string payloads, padding, and signatures are structured in raw memory.
- **Reusable Real-Mode Console (Task 2):** A real-mode console was implemented using BIOS video interrupts (INT 0x10) to safely output text and parse 8-bit hexadecimal values. The processor state was safely initialized by disabling maskable interrupts before modifying the stack segment and pointer.
- **Robust Multi-Stage Loader (Task 3):** A two-stage bootloader was successfully engineered. Stage 1 utilized Extended LBA reads (AH=0x42) to load Stage 2 from the disk. Stage 2 then loaded the kernel, validated the KRNL magic bytes, enabled the A20 address line, loaded a custom Global Descriptor Table (GDT), and performed a far jump to successfully transition the CPU into 32-bit protected mode.
- **Protected-Mode Mini Kernel (Task 4):** A freestanding 32-bit C kernel was compiled, linked, and executed. Custom VGA text and PS/2 keyboard drivers were implemented by directly interacting with memory address 0xB8000 and hardware I/O ports 0x60 and 0x64. An interactive shell was built that successfully executed all required diagnostic and hardware-manipulation commands (e.g., regs, peek, color, reboot, and halt).

Conclusion

In conclusion, this lab provided profound, hands-on insights into low-level system architecture and bare-metal programming. By stripping away the abstractions of modern operating systems and standard C libraries, the assignment successfully demonstrated how software wrestles control of the hardware upon startup. The successful implementation of a multi-stage bootloader and an interactive protected-mode shell proved that hardware manipulation, memory validation, and basic driver development can be seamlessly achieved from scratch.